

JAVA 17 OCP · YDS STUDY SERIES

OCP Java 17 · İş Gününe Uyan Çalışma Planı

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu planın amacı aynı oturumda bir Java kuralını anlamayı, o kuralın İngilizcesini çözmeni ve kısa bir soruyla bilgini kontrol etmeni sağlamaktır. Başlangıç temposu **günde 25–30 dakika** olarak önerilmiştir. Süre dolduğunda kaldığın başlığı yaz; bir üniteyi tek oturumda bitirmeye çalışma. Ünite girişlerindeki rotalar konu dilimlerini gösterir; uzun bir dilimi birkaç güne yayabilirsin.

İçindekiler

- 1 [Bugün ne yapacağım?](#)
- 2 [Bir İngilizce cümleyi nasıl çözerim?](#)
- 3 [Bilmediğim kelimeyi nasıl seçerim?](#)
- 4 [Sorulardan nasıl öğrenirim?](#)
- 5 [Tekrar ve ilerleme kaydı](#)
- 6 [İlk hafta ve ünite geçişi](#)

Bugün ne yapacağım?

Ünitenin `README.md` dosyasındaki ilk tamamlanmamış konu dilimini seç. O dilimin ana ders başlığı, kelimeleri, grammar yapısı ve kaynak soru numaraları aynı rotada gösterilir. Yardımcı PDF'lerin tamamını her gün açman gerekmez.

Süre	Yapılacak iş	Oturumdan kalan somut çıktı
0–3 dk	Önceki konuyu kaynak kapalıyken hatırla	Bir kural ve önceki yanışının nedeni
3–13 dk	Ana dersten 1–3 English/Türkçe paragraf çifti çalış	Kuralın kendi cümlele açıklaması
13–18 dk	En çok 3–5 kelime ve bir cümle yapısını çöz	Bir doğal çeviri ve bir yeni örnek
18–26 dk	O konuya bağlı 1–2 soruyu çöz, ardından cevapları incele	Derleme / exception / çıktı kararı ve gerekçesi
26–30 dk	Yanışını ve sıradaki başlığı kaydet	Bir tekrar tarihi ve ertesi günün başlangıcı

Yoğun gün · 10 dakika: Önceden öğrendiğin bir kuralı anlat, üç kelimeyi hatırla, eski bir yanışını yeniden çöz. Yeni konu eklemek zorunda değilsin.

Daha uzun gün · 50–60 dakika: Kısa bir ara verip ikinci konu dilimine geç veya haftalık yanışlarını çöz. Süreyi yalnızca yeni sayfa okumaya ayırma.

Kontrol noktası: Türkçeyi okuyunca anlamak ile İngilizceyi tek başına çözmek farklı işlerdir. Önce English bloğunu dene; Türkçeyi sonra aç.

Bir İngilizce cümleyi nasıl çözerim?

Her sözcüğü sırayla çevirmeden önce cümlelerin iskeletini bul:

- 1 Çekimli fiili ve varsa modal'ı işaretle: **is, can, must, does not**.
- 2 “Kim/ne?” sorusuyla özneyi bul. Emir cümlesinde **you** yazılmamış olabilir.
- 3 **if, unless, because, although, whereas** gibi bağlantıları ayır.
- 4 **not, only, at least, all, any** gibi sonucu değiştiren sözcükleri bul.
- 5 Önce ana yargıyı, sonra koşul/neden/karşıtlık ilişkisini doğal Türkçeye aktar.
- 6 Java kuralı değişti mi diye kontrol et: olasılık zorunluluğa dönüşmüş mü?

Teknik terimi koruyarak çevir

override, overload, interface, thread veya **exception** gördüğünde terimi Türkçeleştirmeye çalışmak yerine cümledeki görevini çöz. Örneğin “This method may throw an exception” cümlesi “Bu method bir exception fırlatabilir” olur. Burada öğrenilecek ayrım **may** sözcüğünün olasılık bildirmesidir; cümle method'un **her çağrıda** exception fırlattığını söylemez.

Kod adını genel sözcükten ayır: **File** bir API adı olarak aynen kalır; “save the file” ise “dosyayı kaydet” diye çevrilebilir. Bir terimi zaten biliyorsan sırf İngilizce olduğu için kelime listene ekleme. Ünitenin sözlüğünde özellikle karıştırdığın teknik anlamı ve sınav sonucunu değiştiren kalıpları çalış.

Birlikte çözüm · Unit 01

Unit 01 grammar notundaki örnek:

English: Assume the classes are in different files unless the question says otherwise.

Türkçe: Soruda aksi belirtilmedikçe sınıfların farklı dosyalarda olduğunu varsay.

Cümle parçası	Görevi	Anlama katkısı
Assume	Emir; özne örtük you	Varsay
the classes are in different files	Varsayılan yargı; örtük that ile nesne cümlecisi	Sınıfların farklı dosyalarda olduğunu
unless	Olumsuz koşul bağlantısı	Aksi belirtilmedikçe
the question says otherwise	Koşul cümlecisi	Soru farklı bir durum belirtirse varsayım değişir

unless koşulu ters çevirmene neden olmasın: soru sessizse **farklı dosyalar** varsayılır. Bu, bütün Java sınıflarının her zaman farklı dosyalarda bulunması gerektiğini söylemez; örnekte bir **soru çözme varsayımı** kurulmuştur.

Şimdi sen: Türkçeyi kapat, cümleyi yeniden çevir. Ardından **unless** parçasını **if the question does not say otherwise** ile değiştirerek aynı anlamın korunduğunu açıkla.

Ayrıntı: [Unit 01 grammar](#) ve [Unit 01 vocabulary](#).

Bilmediğim kelimeyi nasıl seçerim?

Ünite sözlüğü bir aday havuzudur; içindeki her sözcüğü bilmediğin varsayılmaz. Yeni sözcüğü kendi kendine şu şekilde değerlendir:

Durum	Kontrol	Sonraki adım
0 · Yeni	Cümledeki anlamını açıklayamıyorum	Sözlükte bağlamı ve örneği oku
1 · Tanıyorum	Görünce anlıyorum, kendim kullanamıyorum	Türkçeyi kapatıp anlamı söyle; yeni bir Java cümlesi kur
2 · Kullanıyorum	Anlamı ve yakın sözcükten farkını açıklayabiliyorum	Daha sonraki bir oturumda yeniden kontrol et

Bir oturumda **3–5 anlamlı madde** seç. Bu sayı başlangıç için iş yükü önerisidir. Önceliği soruyu yanlış anlamana neden olan kelime ve kalıplara ver. Örneğin *eligible* ile *guaranteed*, uygunluk ile garanti arasındaki farkı taşır. Kelimenin anlamını biliyorsan yeni kart açmak yerine yanlış anladığın bağlamı mevcut sözlük maddesine ekle.

Kayıt yeri her zaman çalıştığın ünitenin *vocabulary.md* dosyasıdır. Yeni grammar açıklaması da o ünitenin *grammar_notes.md* dosyasına gider. Böylece tekrar sırasında sözcüğü gördüğün Java konusuna geri dönebilirsin.

Küçük alışkanlık: Her yeni kelime için bir sınır sorusu sor: “Bu kelime neyi garanti etmiyor?” Örneğin garbage collection için uygun olmak, nesnenin hemen silineceğini garanti etmez.

Sorulardan nasıl öğrenirim?

Önce soruyu cevapla, sonra açıklamayı aç. Kaynak kitaptaki **Review Questions** ile bizim **özgün practice quiz** sorularımız ayrıdır. Ana notun Appendix bölümündeki kaynak cevapları kitap sorularına; practice quiz'in cevap bölümü ise yalnız o quiz'e aittir. *Official Answer* başlıkları kitabın cevap bölümünü izlemek içindir; gerçek sertifika sınavından alınmış soru anlamına gelmez.

Her kod sorusunda üç ayrı kontrol yap:

- Does not compile / derlenmez:** Kodu durduran ilk kuralı ve satırı belirt.
- Runtime exception / çalışma zamanı istisnası:** Derleniyorsa yürütme sırasında hangi işlem hangi istisnayı oluşturuyor? Öncesinde çıktı var mı?
- Output / çıktı:** Akış tamamlanıyorsa sırayı, boşlukları ve satır sonlarını izle. Concurrency sorularında tek bir çıktı garanti edilmeyebilir.

Sonsuz döngü veya deadlock varsa programın tamamlanmaması da ayrı bir olası sonuçtur. Her derlenen kodun ya istisna atacağını ya da biteceğini varsayma.

Yanışını bir cümleye indir

Yanışın türü	Yazılacak not	Sonraki deneme
Java kuralı	“Yerel değişkene varsayılan değer verildiğini sandım.”	Yerel değişken ile field'ı karşılaştır
Kod izleme	“Post-increment'in eski değeri kullandığını atladım.”	Değer ve yan etkiyi ayrı sütunlara yaz
Kelime	“ <i>at least</i> ifadesini tam sayı sınırı sandım.”	“En az” ile “tam olarak” farkını örnekendir
Grammar	“ <i>unless</i> koşulunun yönünü ters çevirdim.”	<i>if ... not</i> dönüşümü yap
Dikkat	“İki seçenek istenirken birini işaretledim.”	Seçim yönergesini çözümden önce işaretle

Bir doğru seçenek ve en güçlü yanlış seçenek için ayrı birer neden yaz. Doğru harfi hatırlamak tek başına yeterli değildir. Çözümü gördükten sonra örneğin bir sabitini veya modifier'ını değiştirip sonucun nasıl değişeceğini tahmin et; doğrulamadan bu tahmini ders notuna kesin bilgi olarak ekleme.

Tekrar ve ilerleme kaydı

İlk öğrenmeden **1, 3, 7 ve 14 gün sonra** kısa kontrol yapmayı başlangıç takvimi olarak kullan. Bunlar herkes için kanıtlanmış en iyi aralıklar değildir; iş yoğunluğuna ve hatırlamaya göre değişir. Kaçırdığın günü telafi etmek için yeni konuları üst üste yığma; en eski yanlışından başla.

Tekrarda sıralama: **kaynak kapalı dene** → **cevabı kontrol et** → **kısa düzeltme yaz** → **yeni tarih belirle**. Yanlışsa daha yakın bir tarihte yeniden dene; kolay ve gerekçeli doğruysa arayı aç.

Ünite içinde kullanacağın kayıt şablonu

Bu küçük tabloyu çalıştığın ünitenin giriş sayfasının sonuna kendi kayıtların için kopyalayabilirsin. Henüz bilmediğin sözcük veya tamamladığın çalışma senin adına işaretlenmemiştir.

Tarih	Konu / soru no	Bilmediğim kelime / yapı	Yanışımın nedeni	Yeniden deneme
...	...	...	...	...

İlk hafta ve ünite geçişi

İlk üniteyle başlayacaksan günleri şöyle kullan:

- **İlk iş günü:** Unit 01 girişinden ilk konu dilimini aç; 30 dakikalık akışı dene.
- **İkinci iş günü:** Dünkü kuralı ve kelimeleri hatırla; aynı dilimi tamamla veya sıradaki dilime geç.
- **Sonraki iş günleri:** Birikmiş kısa tekrarları önce yap; sonra kaldığın başlıktan devam et. Bir konu zor geldiyse yeni dilime geçmek zorunda değilsin.
- **Hafta sonu, uygunsa 40–60 dakika:** O hafta çalıştığın konuların kaynak sorularını karışık çöz ve yanlışların ortak nedenini bul.

Bir konu diliminden geçmek için kendine şu üç kanıt ara:

- Java kuralını notsuz açıklayabiliyorum ve bir istisnasını/sınırını biliyorum.
- Çalıştığım cümlelerin ana fiilini, öznesini ve anlam bağlantısını gösterebiliyorum.
- İlgili soruda doğruyu ve en güçlü yanlış seçeneği gerekçelendirebiliyorum.

Ünite sonunda practice quiz'i çöz. İki farklı günde benzer başarı sağlamak, aynı anda cevap anahtarını ezberleyerek yüksek puan almaktan daha anlamlı bir kişisel kontrol sağlar. Buradaki ölçütler çalışma önerisidir; OCP veya YDS geçme puanı ya da başarı garantisi değildir.

Yöntemin dayanağı

Bilgiyi kaynağa bakmadan hatırlama denemeleri, çalışmalarda gecikmeli hatırlamayı desteklemiştir. Bu nedenle kısa soruları yalnız ölçmek için değil, öğrenme oturumunun parçası olarak kullanıyoruz. [Roediger ve Karpicke, 2006](#).

Tekrarları zamana yaymanın yararı ile belirli bir aralık dizisinin üstünlüğü ayrı konulardır. Önerilen 1/3/7/14 takvimi pratik bir başlangıç düzenidir. [Karpicke ve Bauernschmidt, 2011](#).

Ünite bağlantıları için [Çalışma Merkezi](#) sayfasına dön.